

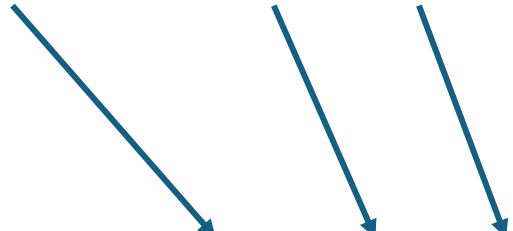
MATLAB

Submatrices

A matrix can be indexed using another matrix, to produce a subset of its elements:

$a = [100 \ 200 \ 300 \ 400 \ 500 \ 600 \ 700]$ $b = [3 \ 5 \ 6]$

$c = a(b):$



300 500 600

The diagram illustrates the indexing process. Three blue arrows originate from the elements 300, 500, and 600 in the array 'a' and point down to the corresponding elements in the resulting array 'c'. This visualizes how the values in 'b' are used to select specific elements from 'a' to form 'c'.

Submatrices

To get a subsection of a matrix, we can produce the index matrix with the colon operator:

```
prints          a(2:5)
```

```
ans = 200 300 400 500
```

- This works in 2-D as well, e.g. `c(2:3, 1:2)` produces a 2 x 2 submatrix.
- The rows and columns of the submatrix are renumbered.

$+$	addition
$+A$	Single plus
$-$	subtraction
$-A$	unary minus
$.*$	Element-wise multiplication
$*$	Matrix multiplication
$./$	Element-wise right division
$/$	Matrix right division
$.\backslash$	Element-wise left division
$\backslash$	Matrix left division (also called <i>backslash</i>)
$.^{\wedge}$	Element-wise exponentiation
$^{\wedge}$	Matrix power
$.'$	transposition
$'$	conjugate complex transpose

Subtract vector from matrix

To simplify vector-matrix operations, use implicit expansion on dimension functions such as `sum`, `mean`, `etc.min`

For example, compute the mean for each column of a matrix, and then subtract the mean from each element.

```
A = magic(3)
```

```
C = mean(A)
```

```
A - C
```

MATLAB Basic

Create a diagonal matrix

```
v = [2 1 -1 -2 -5];
```

```
D = diag(v)
```

```
D1 = diag(v,1)
```

eye(m,n) unit diagonal

Adding row and column vectors

Row vectors and column vectors are compatible in size, and performing operations on these two vectors returns a matrix as a result.

$$a = [1 \ 2 \ 3 \ 4]$$

$$b = [5; 6; 7]$$

$$a + b$$

Expressing polynomials

MATLAB® represents polynomials as row vectors containing coefficients sorted in descending order by power. For example, the following three-element vector

$$p = [p_2 \ p_1 \ p_0];$$

Express the following polynomial:

$$p(x) = p_2x^2 + p_1x + p_0.$$

Second degree polynomial $p(x) = x^2 - 4x + 4$ Create a vector representing .

$$p = [1 \ -4 \ 4];$$

Polynomial $p(x) = 4x^5 - 3x^2 + 2x + 33$ Create a vector representing .

$$p = [4 \ 0 \ 0 \ -3 \ 2 \ 33];$$

polyval can use functions to evaluate the polynomial at specific values.

polyval(p,2)

roots(p)

MATLAB Basic

Vector Products

Consider & generate two random 3*1 vectors, A & B
`A=rand(3,1)` , `B=rand(3,1)`

Dot Product

- Check Dimensions
 - `A*B`
 - `A*B'`
 - `A'*B`
- Use MATLAB command
`>> C=dot(A,B)`

Plus, Minus

`C = A + B;`
`D = A - B;`

- `A(2)*B(3)-A(3)*B(2)`
- `A(3)*B(1)-A(1)*B(3)`
- `A(1)*B(2)-A(2)*B(1)`

Cross Product

- By definition
- Use MATLAB command
`>> C=cross(A,B)`

$$C = A \times B = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$
$$= (a_2 b_3 - a_3 b_2) \hat{i} + (a_3 b_1 - a_1 b_3) \hat{j} + (a_1 b_2 - a_2 b_1) \hat{k} .$$

Dot Operator

- For scalar operations, nothing new is needed. Example:
$$a = 5; b = 3; \implies c = a * b \% c = 15$$
- For element operations, a dot must be used before the operator.
- Note: Dot operator not the same as dot product!
- Example:
 - `a = [1 2 3 4];`
 - `b = [5 6 7 8];`
 - `c = a*b`
 - Result: ??? Error using `mtimes`
inner matrix dimensions must agree
- Now, try:
 - `c = a.*b` %notice the dot!
 - Result: `c=[5 12 21 32]`
- Notice what it is doing: $a(1)*b(1)$, $a(2)*b(2)$, etc.

MATLAB Basic

```
A_T = A'; % Transpose of matrix A
```

```
inv_A = inv(A); % Inverse of A (only if A is square and invertible)
```

solves the system of linear equations $A \cdot x = b$.

```
b = [6 ; 7];  
>> A = [1 3; 4 2];
```

```
x = A \ b
```

```
x = inv(A) * b
```

```
x = mldivide(A, B)
```

MATLAB Basic

```
m = 4;  
A = gf([8 1 6;3 5 7;4 9 2],m);  
B = gf(2*eye(size(A)),m);
```

Galois matrix to divide by, specified as an array with Galois field entries.

```
x = A \ B;
```

Data Types: gf

$$\begin{aligned}2x + y + z &= 2 \\ -x + y - z &= 3 \\ x + 2y + 3z &= -10\end{aligned}$$

```
syms xyz
```

```
eqn1 = 2*x + y + z == 2;
```

```
eqn2 = -x + y - z == 3;
```

```
eqn3 = x + 2*y + 3*z == -10;
```

```
[A,B] = equationsToMatrix([eqn1, eqn2, eqn3], [x, y, z])
```

X =

3

1

-5

Xln , x = 3 , y = 1 , z = -5 .

```
X = linsolve(A,B)
```

MATLAB Basic

```
sol = solve([eqn1, eqn2, eqn3], [x, y, z]);  
xSol = sol.x  
ySol = sol.y  
zSol = sol.z
```

```
xSol = 3  
ySol = 1  
zSol = -5
```

```
syms abc x  
eqn = a*x^2 + b*x + c == 0;  
solx = solve(eqn, x)
```

```
solx =  
-(b + (b^2 - 4*a*c)^(1/2))/(2*a)  
-(b - (b^2 - 4*a*c)^(1/2))/(2*a)
```

MATLAB Basic

```
solb = solve(eqn, b)
```

```
solb =
```

```
-(a*x^2 + c)/x
```

Loops

- Loop statements do not need parenthesis. The statements are recognized via tabs.
- 3 general types of loops:
 - If/else/elseif loops
 - For loops
 - While loops
- There is a 4th type, called nested loop, that can be any of the above 3 (and any combinations of them).

MATLAB Basic

If/else/elseif Loops

- The condition must be previously defined!

```
if condition
    statement
end
```

```
if condition
    statement
else
    statement
end
```

```
if condition
    statement
elseif
    statement
else
    statement
end
```

Example

```
x = 5;
if x < 5
    answer = 'x is less than 5!'
elseif x < 10
    answer = 'x is less than 10, but greater than 4!'
else
    answer = 'x is greater than 10!'
end
```

Which loop statement will be triggered?

loops

The typical 'for' loop looks like:

```
for i = 1:6  
    ...  
end
```

Which is the same as:

```
for i = [1 2 3 4 5 6]  
    ...  
end
```

loops

‘for’ loops in MATLAB iterate over matrix elements:

```
b = 0
for i = [ 3 9 17]
    b = b + i;
end
```

Result: 29

Note:

The MATLAB way to write that program would have been:

```
b = sum([ 3 9 17]);
```

Avoid loops if possible !